BML Project 2 report
Gabriel Kiciński

1 Implementation notes

The goal of this project is to implement several up-to-date features in the training algorithm of the Transformer architecture. Using the C4 dataset, we aim to enable efficient training of large models on multiple devices.

1.1 Description of Main Modifications

1. **Logging.** Using the wandb platform we monitor the hyperparameters as well as loss so the experiments are easily tracked. The tracking is performed by one chosen worker. So it reports only its own training loss, the validation loss is calculating by averaging outcomes of workers on the val dataset.

2. **FSDP algorithm.** Imported from `torch.distributed.fsdp`. The model is wrapped at the top level, each GPU will hold a shard of the model parameters according to the method. Process group is initialized via `dist.init_process_group("nccl")` and each process gets its own GPU. This method significantly reduces the memory needed in every machine. `torch.cuda.set_device(dist.get_rank())`.

3. **Mixed Precision** is used inside the FSDP. This ensures FSDP operations use bfloat16 for parameters and reductions, reducing memory usage while maintaining numerical stability.

4. **Data sharding.** The dataset is explicitly sharded per rank. For training, the dataset is shuffled after sharding with `seed + rank` to give each process a unique order. Each GPU worker sees non-overlapping batches.

5. **Warmup-Stable-Decay** was implemented in according to the paper information. It consists of a warmup , a constant hold phase, and a decay. The warmup and decay are linear. The fraction of learning steps in each phase was given in the project description.

6. **SwiGLU.** The feedforward layer was replaced with a SwiGLU variant using SiLU (Swish) activation.

$$\mathrm{FF}_{\mathrm{SwiGLU}}(x) = \big(\mathrm{SiLU}(xW_2) \odot xW_1\big)W_3, \quad \mathrm{SiLU}(z) = z \cdot \frac{1}{1+e^{-z}}$$

2 Optimal number of tokens

Number of model parameters N:

$N =$ token embeddings and outputs params + pos. embeddings + transformer blocks params

$N \approx 2 \times \text{vocab_size} \times d_{\text{model}} + Ld_{model} + \text{num_layers} \times \left(3d_{\text{model}}^2 + d_{\text{model}}^2 + 3d_{\text{model}} \times d_{\text{ff}}\right)$

$d_{\text{ff}} = 4\, d_{\text{model}}$

$N \approx 2 \cdot \text{vocab_size} \cdot d_{\text{model}} + \text{num_layers} \cdot 16\, d_{\text{model}}^2$

$N \approx 2 \cdot 50257 \cdot 256 + 4 \cdot 16 \cdot 256 \cdot 256 = 29925888 \approx 3 \cdot 10^7$

Optimal number of tokens D :

$$D = 20 \cdot N = 6 \cdot 10^8 \text{ tokens}$$

Number of training steps (compute-optimal, extrapolated via Chinchilla scaling):

$$\text{steps} = \frac{D}{\text{batch_size} \times \text{seq_length}} = \frac{6 \cdot 10^8}{256 \cdot 256} = 9150$$

With two GPUs, the optimal number of steps per worker is 4575. I used 4576 to round slightly above.

3 Experiment details

3.1 Learning rate

On the figure we show the plot of the learning rate from the experiment, clearly illustrating the three training phases of the Warmup-Stable-Decay method.

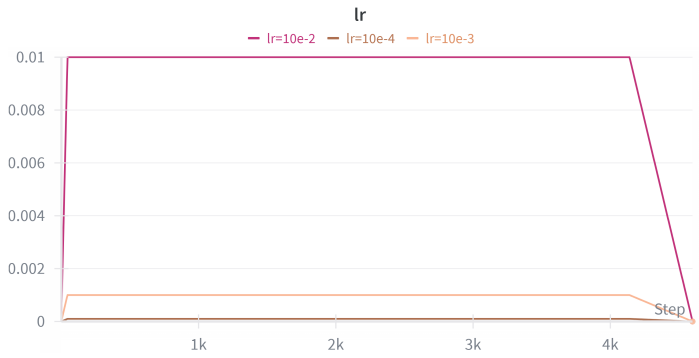


Figure 1: Learning rate versus step number in final experiments.

3.2 Proving multigpu training

Wandb is not compatible with GPU monitoring on the Athena cluster. Multi-GPU training can be observed from the experiment outputs. The workers produce different training losses at each step, confirming data sharding and existence of two workers.

3.3 Losses from experiments

A grid search was conducted using three learning rates: $10^{-2}, 10^{-3}, 10^{-4}$.

3.3.1 Stability

The 10^{-2} learning rate was too aggressive for the chosen model, causing highly unstable training. As shown in Figure X, the training and validation losses spike to high values.



(a) Training loss of all runs



(b) Validation loss of all runs

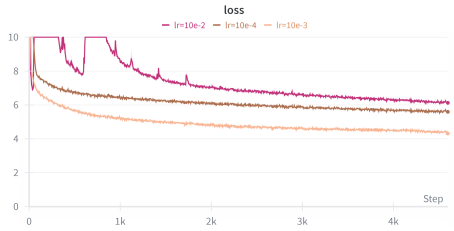
Figure 2: Comparison of all runs, highlighting the instability caused by the largest learning rate.

3.3.2 Performance Comparison

The learning rate of 10^{-3} achieves lower loss than the other two runs. Both the training and validation losses evolve stably and converge. Based on the results shown below, a learning rate of 10^{-3} is the optimal choice.

Learning Rate	Final Validation Loss
10^{-2}	6.16
10^{-3}	**4.40**
10^{-4}	5.63

Table 1: Final validation loss for different learning rates.



(a) Training loss of all runs



(b) Validation loss of all runs

Figure 3: Comparison of all runs, with the scale adjusted for the final loss analysis.